DART PROGRAMMING
CLASS 05

Functions & Object-Oriented Programming

Functions Functions • Parameters Return Types	Function Features Named / Optional Parameters Arrow Functions	OOP Basics Class • Object • Constructor Properties • Methods

Class Purpose

This handout introduces reusable functions and the basic building blocks of Object-Oriented Programming in Dart. Students will learn how to create functions, pass data into them, return results, and build classes and objects using constructors, properties, and methods.

Level: Beginner Language: Dart Focus: Functions + OOP

01. Learning Objectives

- Understand why functions are used to organize and reuse code.
- Create functions with and without parameters.
- Understand return types and return values.
- Use positional, named, and optional parameters correctly.
- Write concise functions using arrow syntax.
- Understand the relationship between a class and an object.
- Create constructors and initialize object properties.
- Use named constructors to create objects in different ways.
- Create properties and methods inside a class.
- Build a simple Dart class using all concepts from this lesson.

By the end of this class, students should be able to write reusable Dart functions and create simple classes with objects, constructors, properties, and methods.

02. What is a Function?

A function is a reusable block of code designed to perform a specific task. Instead of writing the same logic multiple times, we place it inside a function and call it whenever needed.

Why Use Functions?

- Code reusability — write once and use multiple times.
- Better organization — divide a large program into smaller tasks.
- Readability — meaningful names make code easier to understand.
- Maintenance — change logic in one place.

Basic Function Structure

```
returnType functionName() {
  // function body
}
```

Example — Simple Function

```
void greet() {
  print("Hello, Dart!");
}

void main() {
  greet();
  greet();
}
```

Important: Defining a function does not execute it. The function runs when you call it.

03. Parameters

A parameter is a variable listed in a function definition. It allows a function to receive data from the caller.

One Parameter

```
void greet(String name) {
  print("Hello, $name!");
}
```

```
void main() {
  greet("Rahim");
  greet("Karim");
}
```

Multiple Parameters

```
void addNumbers(int a, int b) {
  print(a + b);
}

void main() {
  addNumbers(10, 20);
}
```

Parameter vs Argument

Parameter: variable in the function definition, e.g. int a.

Argument: actual value passed during the call, e.g. 10.

```
void add(int a, int b) { // parameters
  print(a + b);
}

add(10, 20);             // arguments
```

Homework / Practice

1. Create a function that takes a name and prints a greeting.
2. Create a function that takes two numbers and prints their sum.
3. Create a function that takes length and width and prints rectangle area.

Important Questions

1. What is a function?
2. What is the difference between a parameter and an argument?
3. Why are parameters useful?

04. Return Types and Return Values

A function can return a value to the code that called it. The return type tells Dart what type of value the function returns.

void Return Type

Use void when the function performs an action but does not return a value.

```
void printMessage() {
  print("Hello");
}
```

Returning an int

```
int add(int a, int b) {
  return a + b;
}

void main() {
  int result = add(10, 20);
  print(result); // 30
}
```

Returning String and bool

```
String getFullName(String first, String last) {
  return "$first $last";
}

bool isEven(int number) {
  return number % 2 == 0;
}

void main() {
  print(getFullName("Rahim", "Ahmed"));
  print(isEven(10)); // true
}
```

Key rule: If a function has a non-void return type, make sure it returns an appropriate value on every possible execution path.

Homework / Practice

1. Create int square(int number) that returns the square.
2. Create double calculateAverage(int a, int b, int c) that returns the average.
3. Create bool isAdult(int age) that returns true when age is 18 or above.

Important Questions

1. What is a return type?
2. What is the difference between void and int as a return type?
3. Why do we use the return keyword?

05. Named Parameters

Named parameters allow arguments to be passed using parameter names. They make function calls easier to read and allow arguments to be supplied in a different order.

Named Parameter Syntax

```
void createUser({
  required String name,
  required int age,
}) {
  print("Name: $name");
  print("Age: $age");
}

void main() {
  createUser(name: "Rahim", age: 22);
}
```

Named Parameter with a Default Value

```
void greet({
  required String name,
  String country = "Bangladesh",
}) {
  print("$name is from $country");
}

void main() {
  greet(name: "Rahim");
  greet(name: "John", country: "USA");
}
```

06. Optional Parameters

Optional parameters allow the caller to omit an argument. Dart supports optional positional and optional named parameters.

Optional Positional Parameters — []

```
void greet(String name, [String? message]) {
  if (message != null) {
    print("$message, $name");
  } else {
    print("Hello, $name");
  }
}

greet("Rahim");
greet("Karim", "Good morning");
```

Optional Named Parameters — {}

```
void showInfo(String name, {int? age}) {
  print("Name: $name");

  if (age != null) {
    print("Age: $age");
  }
}

showInfo("Rahim");
showInfo("Karim", age: 25);
```

Remember: [] = optional positional parameters. {} = named parameters. required = named argument must be provided.

Homework / Practice

1. Create showProfile(String name, {int? age, String? city}) and print available information.
2. Create calculateTotal(double price, [int quantity = 1]) and return the total.
3. Create createProduct({required String name, double price = 0}) and print the product.

Important Questions

1. What is a named parameter?
2. What does required mean?
3. What is the difference between [] and {} parameters?
4. Why are named parameters useful?

07. Arrow Functions

An arrow function is concise syntax for a function containing exactly one expression. It uses => and does not use a block body.

Normal vs Arrow

```
// Normal
int square(int number) {
  return number * number;
}

// Arrow
int square(int number) => number * number;

int add(int a, int b) => a + b;
bool isEven(int n) => n % 2 == 0;
String greet(String name) => "Hello, $name";
```

Important: Use arrow syntax for short, single-expression functions. Use normal block syntax when multiple statements are needed.

Homework / Practice

1. Convert a cube function into arrow syntax.
2. Create an arrow function that returns the larger of two numbers.
3. Create an arrow function that returns a greeting String.

Important Questions

1. What is an arrow function?
2. When should you use arrow syntax?
3. Can an arrow function contain multiple statements?

08. Class and Object

A class is a blueprint or template. An object is an actual instance created from that class.

Real-World Analogy

Concept	Example
Class	Car blueprint
Object	A specific car
Property	Color, model, speed
Method	Start, stop, accelerate

Basic Class and Object

```
class Student {
  String name = "";
  int age = 0;
}

void main() {
  Student student1 = Student();
```

```
  print(student1.name);
  print(student1.age);
}
```

Class: blueprint. Object: instance of the class.

09. Properties

A property is a variable declared inside a class. It stores information about an object.

```
class Student {
  String name = "";
  int age = 0;
  String department = "";
}

void main() {
  Student student = Student();

  student.name = "Rahim";
  student.age = 22;
  student.department = "CSE";

  print(student.name);
}
```

Use the dot operator . to access or update an object's properties, e.g. student.name.

10. Methods

A method is a function declared inside a class. Methods describe what an object can do.

```
class Student {
  String name = "";

  void introduce() {
    print("My name is $name.");
  }
}

void main() {
  Student student = Student();
  student.name = "Rahim";
  student.introduce();
}
```

Method with Parameters and Return Value

```
class Calculator {
  int add(int a, int b) {
    return a + b;
  }
}

void main() {
  Calculator calculator = Calculator();
  int result = calculator.add(10, 20);
  print(result);
}
```

Important Questions

1. What is a class?
2. What is an object?

3. What is the difference between a property and a method?
4. How do you access an object's property?
5. How do you call an object's method?

11. Constructor

A constructor is a special function used when creating an object. It is commonly used to initialize the object's properties.

Default Constructor

```
class Student {
  String name = "";
  int age = 0;

  Student() {
    print("Student object created");
  }
}

void main() {
  Student student = Student();
}
```

Constructor with Parameters

```
class Student {
  String name;
  int age;

  Student(this.name, this.age);
}

void main() {
  Student student = Student("Rahim", 22);

  print(student.name);
  print(student.age);
}
```

Shortcut: Student(this.name, this.age); automatically assigns constructor arguments to the class properties.

12. Named Constructor

A named constructor gives a class additional ways to create objects. It is useful when objects need different initialization rules.

Example

```
class Student {
  String name;
  int age;

  Student(this.name, this.age);

  Student.guest()
      : name = "Guest",
        age = 0;
}

void main() {
  Student student1 = Student("Rahim", 22);
  Student student2 = Student.guest();

  print(student1.name);
  print(student2.name);
}
```

Named Constructor with Different Roles

```
class User {
  String name;
  String role;

  User(this.name, this.role);

  User.admin(String name)
      : this(name, "Admin");

  User.guest()
      : this("Guest User", "Guest");
}

void main() {
  User u1 = User.admin("Rahim");
  User u2 = User.guest();

  print("${u1.name} - ${u1.role}");
  print("${u2.name} - ${u2.role}");
}
```

Key idea: A normal constructor provides one creation pattern; named constructors provide meaningful alternative creation patterns.

Homework / Practice

1. Create a Book class with title and price and a constructor to initialize them.
2. Create a User class with name and role, then add guest() and admin() named constructors.
3. Create a Product class with a normal constructor and a named constructor called free().

Important Questions

1. What is a constructor?
2. When does a constructor run?
3. What is a named constructor?
4. Why would a class need more than one constructor?

13. Complete Example — Student Class

This example combines properties, a constructor, a named constructor, methods, parameters, return values, and objects.

```
class Student {
  String name;
  int age;
  double marks;

  Student(this.name, this.age, this.marks);

  Student.guest()
      : name = "Guest",
        age = 0,
        marks = 0;

  bool hasPassed() {
    return marks >= 40;
  }

  String getInfo() {
```

```
    return "$name | Age: $age | Marks: $marks";
  }

  void introduce() {
    print("Hello, I am $name.");
  }
}

void main() {
  Student student1 = Student("Rahim", 22, 78);
  Student student2 = Student.guest();

  student1.introduce();
  print(student1.getInfo());
  print(student1.hasPassed());
  print(student2.getInfo());
}
```

Teacher activity: Ask students to identify the class, objects, properties, constructors, named constructor, methods, return types, and method calls in the example.

14. Quick Comparison

Concept	Meaning	Example
Function	Reusable block of code	add(10, 20)
Parameter	Input variable	int a
Return Type	Type of returned value	int / String / bool
Named Parameter	Argument passed by name	name: "Rahim"
Optional Parameter	Argument that may be omitted	[int? age]
Arrow Function	Short single-expression function	add(a,b) => a+b
Class	Blueprint for objects	class Student {}
Object	Instance of a class	Student()
Property	Data stored in object	student.name
Method	Function inside class	student.introduce()
Constructor	Initializes object	Student(this.name)
Named Constructor	Alternative creation	Student.guest()

15. Final Homework — Class Assignment

Complete these tasks independently. For each task, first write the logic, then implement and test it with at least two inputs.

1. Write a function that accepts two numbers and returns their sum.
2. Write a function that accepts a number and returns whether it is even.
3. Write a function with named parameters for name, age, and city.
4. Write a function with an optional positional parameter.
5. Convert three normal one-expression functions into arrow functions.
6. Create a Car class with brand, model, and year properties.
7. Add a constructor to initialize all Car properties.

8. Add a method displayInfo() that prints the car information.
9. Add a named constructor called Car.electric() that creates an electric car.
10. Create a BankAccount class with balance property and deposit() / withdraw() methods.
11. Create a Student class with name, age, and marks, then add a hasPassed() method.

16. Important Viva / Class Questions

1. What is a function and why do we use functions?
2. What is the difference between a function definition and a function call?
3. What is a parameter? What is an argument?
4. What is a return type?
5. What is the purpose of the return keyword?
6. What is the difference between void and a value-returning function?
7. What is a named parameter?
8. What does required mean in a named parameter?
9. What are optional parameters?
10. What is the difference between optional positional and optional named parameters?
11. What is an arrow function?
12. When should you avoid arrow syntax?
13. What is a class?
14. What is an object?
15. What is a property?
16. What is a method?
17. What is a constructor?
18. When does a constructor execute?
19. What is a named constructor?
20. Why are named constructors useful?
21. What is the difference between a normal function and a method?

17. Class Checklist

Concept	Can Explain	Can Code
Functions	☐	☐
Parameters	☐	☐
Return Types	☐	☐
Named Parameters	☐	☐
Optional Parameters	☐	☐
Arrow Functions	☐	☐
Class & Object	☐	☐

Concept	Can Explain	Can Code
Constructor	☐	☐
Named Constructor	☐	☐
Properties	☐	☐
Methods	☐	☐

Teacher Tip: Before running each example, ask students to predict the output. For OOP examples, ask them to identify the class, object, properties, constructor, named constructor, and methods.